

Ridex Backend Project Guide

Simple Hinglish explanation • Rider App + Captain App + Admin + Ride + Payment

Project idea: Ridex ek Uber/Rapido type ride-booking backend hai. Rider ride book karta hai, Captain/Driver accept karta hai, admin driver verify karta hai, aur completed ride ka payment Razorpay se hota hai.

1. Project ka simple overview

Is project mein teen main users hain:

- **Rider/User:** Ride book karta hai.
- **Captain/Driver:** Ride accept karke pickup aur drop complete karta hai.
- **Admin:** Driver documents/details review karke verify ya reject karta hai.

Rider App → User Login → Ride Request → Captain gets request
Captain App → Driver Login → Register → Admin Verify → ONLINE → Accept Ride
Admin Panel → Pending Driver List → Verify / Reject
Payment → Razorpay Order → Checkout → Verify / Webhook → SUCCESS

2. Database ka simple structure

Table	Kaam	Important data
users	Rider app ke accounts	Mobile, name, email, role, profile
drivers	Captain app ke accounts	Mobile, name, license, verification, online status, GPS
vehicles	Driver ki gaadi	Bike/Auto/Cab, number, model, color
locations	Rider ke saved addresses	Home, Work, Other, latitude, longitude
rides	Har ride ka complete record	Pickup, drop, fare, rider, driver, ride/payment status
payments	Razorpay transaction record	Order ID, payment ID, amount, status

Important design: users aur drivers alag tables hain. Dono ka alag OTP login aur alag JWT token hai. Driver ki vehicle vehicles table se linked hai; ride mein rider aur driver dono record hote hain.

3. JWT aur login kaise work karte hain

Rider/User login

1. Rider mobile number deta hai aur OTP receive karta hai.
2. OTP valid ho to naye mobile par users table mein new rider account create hota hai.
3. Backend User JWT token return karta hai.
4. Token ke saath rider profile, locations aur ride APIs use karta hai.

Example: Rahul Rider app mein 9876543210 se login karta hai. Token milne ke baad woh profile update aur ride booking kar sakta hai.

Captain/Driver login

1. Captain Driver OTP API use karta hai.
2. OTP valid ho to drivers table mein account create/find hota hai.
3. Backend Driver JWT token return karta hai.
4. Driver profile incomplete ho to registration form fill kiya jata hai.

Example: Amit Captain app mein 9630503547 se login karta hai, Driver JWT leta hai aur phir license/vehicle details submit karta hai.

Security: User JWT se driver APIs call nahi hoti aur Driver JWT se rider APIs call nahi hoti. Isse account separation secure rehta hai.

4. User/Rider module

Functionality	Kaise kaam karti hai	Example
Send OTP	Mobile validate karke OTP generate karta hai.	Rider 9876543210 enter karta hai.
Verify OTP	OTP valid ho to rider login aur JWT return hota hai.	New user automatically create ho sakta hai.
Get profile	Logged-in rider ki profile return hoti hai.	Rahul apna registered name/email dekhta hai.
Update profile	Name, email, gender, DOB aur image update hoti hai.	Rahul name "Rahul Sharma" save karta hai.
Saved location	Home/Work/Other address save, update, list aur delete hota hai.	Rahul "Home: MG Road, Pune" save karta hai.

5. Driver/Captain module

1. Captain OTP login karke Driver JWT leta hai.
2. Captain full name, license number aur vehicle details submit karta hai.
3. Initial status **INACTIVE** aur verification **false** hoti hai.
4. Admin verify karega.
5. Verify hone ke baad status **OFFLINE** hota hai.
6. Captain "Go Online" karega to status **ONLINE** ho jayega.
7. ONLINE Captain current latitude/longitude bhejta hai.

Example: Amit ne BIKE register ki. Amit sirf BIKE rides dekh aur accept kar sakta hai; CAB ride accept karne par vehicle mismatch error aayega.

```
INACTIVE → Admin verification pending  
OFFLINE → Verified, but ride requests nahi le raha  
ONLINE → Ride requests receive kar sakta hai  
ON_RIDE → Already ek accepted ride par hai  
BLOCKED → Rejected/blocked driver
```

6. Admin driver verification

1. Admin user login karke Admin JWT leta hai.
2. Admin pending drivers ki paginated list dekhta hai.
3. License, name aur vehicle details review karta hai.
4. Verify karne par driver verified aur OFFLINE ho jata hai.
5. Reject karne par driver BLOCKED ho jata hai aur rejection reason save hota hai.

Example: License valid ho to Amit verify hoga. Invalid license ho to admin "License document invalid" reason ke saath reject karega.

7. Ride booking module — complete flow

Step 1: Rider ride request banata hai

Rider vehicle type, pickup GPS aur drop GPS send karta hai. Backend rider authentication, active-ride check, GPS distance calculation aur fare calculation karta hai. Phir ride REQUESTED status ke saath save hoti hai.

Example: Rahul MG Road se Hinjewadi tak CAB request karta hai. Distance 14.7 km aur fare ₹289.60 aa sakta hai. Is time driverId null hona normal hai, kyunki abhi kisi driver ne accept nahi kiya.

Step 2: Nearby driver ko request milti hai

System sirf verified ONLINE drivers ko choose karta hai jinki vehicle type match karti hai, jinki GPS location present hai, aur jo pickup se maximum 10 km ke andar hain. Eligible Captain ko WebSocket se real-time ride request milti hai.

Step 3: Captain accept karta hai

1. Captain request accept karta hai.
2. Ride status DRIVER_ASSIGNED hota hai.
3. Ride mein driver name, mobile aur ID add hote hain.
4. Driver status ON_RIDE ho jata hai.

Step 4: Ride status updates

```
REQUESTED
↓ Driver accepts
DRIVER_ASSIGNED
↓ Driver reaches pickup
DRIVER_ARRIVED
↓ Driver starts trip
RIDE_STARTED
↓ Driver completes trip
RIDE_COMPLETED
```

Rider sirf REQUESTED ya DRIVER_ASSIGNED stage par cancel kar sakta hai. Ride start hone ke baad cancellation allowed nahi hai.

Fare calculation

Vehicle	Base fare	Per km rate	Minimum fare
BIKE	₹25	₹8/km	₹30

AUTO	₹25	₹12/km	₹30
CAB	₹25	₹18/km	₹30

Example: 14.7 km CAB ride: ₹25 + (14.7 × ₹18) = ₹289.60.

8. Real-time WebSocket ride notifications

Earlier ride request sirf database mein save hoti thi. Ab eligible drivers ko real-time WebSocket notification milti hai.

1. Captain app WebSocket server se connect karta hai.
2. Connection par Driver JWT deta hai.
3. Captain sirf apne private driver topic ko subscribe kar sakta hai.
4. Rider new ride create karega.
5. Eligible driver ko immediately ride data push hoga.

WebSocket endpoint: /ws

Driver topic: /topic/driver/{driverId}/ride-requests

Example: Driver ID 2 ka Captain app apne topic ko subscribe karega. Agar woh eligible CAB driver hai, Rahul ki new CAB request turant app par aa jayegi.

9. Payment module — Razorpay flow

1. Driver ride complete karta hai; payment status PENDING rehta hai.
2. Rider payment order create karta hai.
3. Backend ride fare ke amount ka Razorpay order create karta hai.
4. Frontend Razorpay Checkout open karta hai.
5. Razorpay order ID, payment ID aur signature return karta hai.
6. Frontend backend payment verify API call karta hai.
7. Backend signature verify karke payment aur ride ko SUCCESS karta hai.

Security rule: Random payment ID/signature se success nahi milega. Backend Razorpay cryptographic signature check karta hai. Sirf actual Razorpay payment response valid hoga.

Payment restrictions: Sirf ride owner pay kar sakta hai, sirf completed ride pay hoti hai, aur already successful payment wali ride dobara pay nahi hoti.

10. Razorpay Webhook

Webhook Razorpay server se backend ko direct notification bhejta hai. Yeh important hai kyunki user payment ke baad app close kar sakta hai aur frontend verify call miss ho sakti hai.

Razorpay payment event → POST /api/v1/payment/webhook → Signature verify → Payment status update

Razorpay event	Project action
payment.captured	Payment SUCCESS, ride payment SUCCESS
payment.failed	Payment FAILED, ride payment FAILED
refund.created	Payment REFUNDED, ride payment REFUNDED

Razorpay Dashboard mein public webhook URL aur webhook secret configure karna compulsory hai. Localhost test ke liye ngrok jaisa public tunnel use hota hai.

11. Main API groups

Module	Available functions
User Auth	Send OTP, Verify OTP
User Profile	Get profile, Update profile
Saved Locations	Create, List, Update, Delete
Driver Auth	Driver Send OTP, Driver Verify OTP
Driver	Register, Profile, Online/Offline, GPS update
Admin Drivers	Pending list, Details, Verify, Reject
Rider Rides	Request, Details, Cancel, History
Driver Rides	Available requests, Accept, Arrived, Start, Complete, History
Payments	Create order, Verify, Ride payment, History, Webhook

12. Features completed

- ✓ User OTP login, JWT, profile and saved addresses
- ✓ Separate Driver OTP login and Driver JWT
- ✓ Driver registration, license and vehicle details
- ✓ Admin pending driver list, verification and rejection
- ✓ Driver ONLINE/OFFLINE and GPS update
- ✓ Ride request, distance and fare calculation

- ✓ Driver accept, arrived, start and complete lifecycle
- ✓ Rider and driver ride history with pagination
- ✓ Razorpay order creation, signature verification and webhook updates
- ✓ Real-time nearby driver ride notifications through WebSocket

13. Features still remaining

- • Firebase FCM notification for background/closed Captain app
- • Live map tracking between rider and driver during ride
- • Automatic driver assignment/dispatch algorithm
- • Wallet, add money and wallet payment
- • Driver earnings, commission and payout module
- • Ratings and reviews after ride completion
- • Cancellation charges and automatic refund initiation
- • Real SMS provider integration such as MSG91 or Twilio
- • Admin frontend dashboard
- • Automated tests, database migrations and deployment setup

14. Final end-to-end example

1. Rahul Rider app mein OTP login karta hai.
2. Rahul CAB ride request karta hai.
3. Backend fare calculate karke ride REQUESTED save karta hai.
4. Eligible online CAB Captains ko WebSocket notification milti hai.
5. Amit ride accept karta hai; Rahul ko Amit ki details dikhne lagti hain.
6. Amit arrived, start aur complete status update karta hai.
7. Rahul Razorpay Checkout se payment karta hai.
8. Backend payment signature verify karke SUCCESS karta hai.
9. Razorpay webhook server-to-server final payment status confirm karta hai.
10. Rahul aur Amit dono completed ride history dekh sakte hain.

Document generated for Ridex Backend project. API credentials aur secrets kabhi public share nahi karne chahiye.